Back Pain Relief App – Quality Review

2026-03-27 · Ionic React + Capacitor (Android / iOS) / Vite

Executive Summary

The Ionic React/Capacitor app is architected with a single AppProvider workflow, deterministic routine generation, and localized data that keep the UX consistent across languages. The current build is ready for cross-platform deployment, but three blocking issues (static asset paths, workout timer completion reporting, and notification listener cleanup) must be resolved before production release.

Key Findings

Strengths

- AppContext + services persist progress, preferences, and routines centrally while keeping the UI reactive through hooks such as useApp, ensuring a single source of truth and easy testing.
- Deterministic routine generation, coupled with translated exercise data and framer-motion styling, delivers a polished, multilingual experience that feels native on mobile platforms.

Weaknesses

- Every exercise image URL is prefixed with /public, so Vite serves the files at /exercise/...; the current paths resolve to /public/exercise/..., leading to missing hero media in every screen.
- WorkoutTimer never calls onComplete when the countdown finishes, so progress tracking never advances unless the user taps the manual “Done” button.
- Notification listeners are registered without capturing PluginListenerHandle or removing them, which duplicates navigation callbacks after each AppProvider mount / hot reload.

Code Issues

• Broken exercise image references

`src/data/exercises.ts:4-24 – All records hard-code /public/exercise/... but Vite exposes public assets at /exercise/...; the current URLs resolve to /public/exercise and 404 in dev/production, leaving cards/media empty.`

• WorkoutTimer never triggers completion callback

`src/components/WorkoutTimer/WorkoutTimer.tsx:34-53 – The countdown stops and marks itself complete without calling onComplete, so completeExercise (and streak updates) never run unless the user taps the manual completion button.`

• Notification listeners leak on re-mount

`src/context/AppContext.tsx:67-76 + src/services/notifications.ts:155-166 – addListeners registers LocalNotifications.addListener but does not capture/remove the PluginListenerHandle. Re-initializing AppProvider or hot reloading stacks callbacks, leading to duplicated navigation calls and increased memory use.`

Recommendations (prioritized)

1 1. Fix the exercise image paths by pointing to /exercise/... (drop the /public prefix) or load assets via import.meta.url so every platform can resolve the files.

2. Have WorkoutTimer call onComplete when the countdown reaches zero (and optionally when it is reset via the manual Done button) so completeExercise runs automatically and streak logic remains accurate.

3. Store the PluginListenerHandle returned by LocalNotifications.addListener and call .remove() in AppProvider cleanup. While there, prefer router navigation instead of setting window.location.href directly.

4. Expand coverage for routine generation and notification flows: unit-test generateDailyRoutine + shouldRegenerateRoutine, and smoke-test Schedule/Cancel reminder usage to guard regressions.

Actionable Fixes

1. Update src/data/exercises.ts so each image reference is either new URL(/exercise/...) or uses import.meta.url; run npm run dev to verify the exercise thumbnail and workout hero render.

2. In src/components/WorkoutTimer/WorkoutTimer.tsx, call onComplete?.() inside the setInterval branch when prev <= 1, and consider debouncing to avoid double-triggers when the manual “Done” button is also pressed.

3. In AppContext, await notificationService.addListeners, store the PluginListenerHandle, and return a cleanup callback that removes the listener. Use IonRouterOutlet context or history.push instead of window.location.href to keep routing controlled.